Build: Stability Engine

Objective

Create a deterministic **Stability Engine** that answers:

“Can this business operate consistently without disruption?”

It reads:

- revenue history
- customers
- customer concentration
- contracts
- dependencies
- key-person risk
- processes
- capacity
- backlog

It writes structured output for:

- Master Aggregator
- Fragility Index
- Scenario Simulation
- Adaptive Scenario Intelligence
- AI Narratives
- PDF Reports

Recommended Folder

`/src/lib/engines/stability`

Suggested structure:

```
/src/lib/engines/stability
 ├── types.ts
 ├── rules.ts
 ├── stability-engine.ts
 ├── stability-engine.test.ts
 └── index.ts
```

1. Input Contract

```
export interface StabilityEngineInput {
 businessId: string;

  revenue: {
    revenueLast6Months: number[];
    recurringRevenuePct?: number;
    contractRevenuePct?: number;
  };

  customers: {
    totalCustomers: number;
    top1CustomerRevenuePct: number;
    top3CustomersRevenuePct?: number;
    repeatCustomerRate?: number;
  };

  operations: {
    keyPersonDependency: boolean;
    hasBackupForKeyPerson?: boolean;
    numberOfKeyStaff?: number;
    processesDocumented?: boolean;
    automationLevel?: "low" | "medium" | "high";
  };

  capacity: {
    currentCapacityUnits: number;
    currentUtilisationPct: number;
    maxCapacityUnits?: number;
```

```
    backlogVolume?: number;
  };

  metadata?: {
    source?: "manual" | "imported" | "integration";
    confidenceLevel?: "LOW" | "MEDIUM" | "HIGH";
    rulesetVersion?: string;
  };
}
```

2. Output Contract

```
export interface StabilityEngineOutput {
  engine: "stability";
  version: string;
businessId: string;

  metrics: {
    revenueMean: number | null;
    revenueStdDev: number | null;
    predictabilityRatio: number | null;
    top1CustomerRevenuePct: number;
    top3CustomersRevenuePct: number | null;
    totalCustomers: number;
    currentUtilisationPct: number;
    backlogVolume: number;
  };

  scores: {
    predictabilityScore: number;
    customerScore: number;
    dependencyScore: number;
    capacityScore: number;
    stabilityScore: number;
  };

  classification: {
```

```
  scenario:
    | "HIGHLY_STABLE"
    | "STABLE_WITH_LIMITS"
    | "FRAGILE_STRUCTURE"
    | "UNSTABLE"
    | "CUSTOMER_DEPENDENCY_CRITICAL"
    | "OPERATIONAL_SINGLE_POINT_FAILURE"
    | "CAPACITY_OVERLOAD";
  label: string;
};

riskFlags: Array<{
  code: string;
  severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  explanation: string;
  recommendedAction: string;
  priorityScore: number;
}>;

recommendations: Array<{
  type: string;
  priority: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
  text: string;
  expectedImpact?: string;
}>;

interpretation: {
  headline: string;
  driverSummary: string;
  businessInterpretation: string;
  riskOutlook: string;
  recommendedActions: string[];
};

confidence: {
  level: "LOW" | "MEDIUM" | "HIGH";
  note: string;
  penaltyApplied: boolean;
};
```

```
  trace: Record<string, unknown>;
}
```

3. Core Calculations

Use the shared Computation Library.

Revenue Mean

```
revenue_mean = average(revenue_last_6_months)
```

Revenue Standard Deviation

```
revenue_std = standard_deviation(revenue_last_6_months)
```

Predictability Ratio

```
predictability_ratio =
revenue_std / revenue_mean
```

Rules:

- If revenue history has fewer than 2 months, return null.
- If revenue mean <= 0, return null.
- If null, apply confidence penalty.

Customer Concentration

Input is already percentage/decimal.

Recommended format:

```
0.55 = 55%
```

If current UI stores 55, normalize to `0.55`.

Capacity Utilisation

Input is already percentage/decimal.

Recommended format:

`0.78 = 78%`

If current UI stores 78, normalize to `0.78`.

4. Scoring Rules

A. Revenue Predictability Score

```
if predictabilityRatio === null => 50
else if predictabilityRatio < 0.3 => 100
else if predictabilityRatio <= 0.6 => 70
else if predictabilityRatio <= 1.0 => 40
else => 10
```

Adjustment:

```
if recurringRevenuePct > 0.5:
  predictabilityScore += 10
  cap at 100
```

B. Customer Concentration Score

```
if top1CustomerRevenuePct < 0.2 => 100
else if top1CustomerRevenuePct <= 0.4 => 70
```

```
else if top1CustomerRevenuePct <= 0.6 => 40
else => 10
```

Penalty:

```
if top3CustomersRevenuePct > 0.7:
  customerScore -= 10
  floor at 0
```

C. Operational Dependency Score

```
if keyPersonDependency === false && processesDocumented === true:
  dependencyScore = 100

else if keyPersonDependency === false:
  dependencyScore = 80

else if keyPersonDependency === true && hasBackupForKeyPerson ===
true:
  dependencyScore = 70

else if keyPersonDependency === true && numberOfKeyStaff &&
numberOfKeyStaff <= 2:
  dependencyScore = 40

else if keyPersonDependency === true && hasBackupForKeyPerson ===
false:
  dependencyScore = 10

else:
  dependencyScore = 40
```

Adjustment:

```
if automationLevel === "high":
  dependencyScore += 10
  cap at 100
```

D. Capacity Score

```
if currentUtilisationPct >= 0.5 && currentUtilisationPct <= 0.75:
  capacityScore = 100

else if currentUtilisationPct > 0.75 && currentUtilisationPct <= 0.9:
  capacityScore = 70

else if currentUtilisationPct > 0.9:
  capacityScore = 40

else:
  capacityScore = 40
```

5. Final Stability Score

```
stability_score =
0.30 × predictability_score +
0.25 × customer_score +
0.20 × dependency_score +
0.25 × capacity_score
```

6. Classification Logic

Override scenarios come first.

```
if top1CustomerRevenuePct > 0.6:
  CUSTOMER_DEPENDENCY_CRITICAL

else if keyPersonDependency === true && hasBackupForKeyPerson ===
false:
  OPERATIONAL_SINGLE_POINT_FAILURE

else if currentUtilisationPct > 0.9 && backlogVolume > 0:
  CAPACITY_OVERLOAD
```

```
else if stabilityScore >= 80:
  HIGHLY_STABLE

else if stabilityScore >= 60:
  STABLE_WITH_LIMITS

else if stabilityScore >= 40:
  FRAGILE_STRUCTURE

else:
  UNSTABLE
```

Labels:

```
HIGHLY_STABLE = "Highly Stable"
STABLE_WITH_LIMITS = "Stable with Limits"
FRAGILE_STRUCTURE = "Fragile Structure"
UNSTABLE = "Unstable"
CUSTOMER_DEPENDENCY_CRITICAL = "Customer Dependency Critical"
OPERATIONAL_SINGLE_POINT_FAILURE = "Operational Single Point of
Failure"
CAPACITY_OVERLOAD = "Capacity Overload"
```

7. Risk Flags

CUSTOMER_CONCENTRATION_RISK

Trigger:

```
top1CustomerRevenuePct > 0.4
```

Severity:

```
top1CustomerRevenuePct > 0.6 ? "CRITICAL" : "HIGH"
```

Action:

Diversify revenue sources and reduce dependency on the top customer.

SINGLE_CUSTOMER_DEPENDENCY

Trigger:

`top1CustomerRevenuePct > 0.6`

Severity:

CRITICAL

Action:

Build alternative revenue sources before expanding fixed commitments.

REVENUE_UNSTABLE

Trigger:

`predictabilityScore <= 40`

Severity:

HIGH

Action:

Increase recurring revenue, improve contract stability, and reduce revenue volatility.

KEY_PERSON_RISK

Trigger:

`keyPersonDependency === true`

Severity:

`hasBackupForKeyPerson === false ? "CRITICAL" : "HIGH"`

Action:

Document key processes, create backups, and reduce dependency on one person.

CAPACITY_CONSTRAINT

Trigger:

`currentUtilisationPct > 0.9`

Severity:

`HIGH`

Action:

Increase capacity, improve efficiency, or manage demand before overload causes delivery failure.

UNDERUTILISATION_RISK

Trigger:

`currentUtilisationPct < 0.5`

Severity:

`MEDIUM`

Action:

Improve sales conversion, increase demand generation, or reduce idle capacity.

TOP3_CUSTOMER_CONCENTRATION

Trigger:

```
top3CustomersRevenuePct !== undefined && top3CustomersRevenuePct > 0.7
```

Severity:

```
HIGH
```

Action:

```
Reduce concentration across the top three customers by expanding the
active customer base.
```

NO_REVENUE_HISTORY

Trigger:

```
revenueLast6Months.length < 2
```

Severity:

```
MEDIUM
```

Action:

```
Add at least 3–6 months of revenue records to improve stability
confidence.
```

8. Recommendations Logic

Generate top 3 recommendations based on risk priority.

Priority order:

1. SINGLE_CUSTOMER_DEPENDENCY
2. CUSTOMER_CONCENTRATION_RISK

3. KEY_PERSON_RISK
4. CAPACITY_CONSTRAINT
5. REVENUE_UNSTABLE
6. TOP3_CUSTOMER_CONCENTRATION
7. UNDERUTILISATION_RISK
8. NO_REVENUE_HISTORY

Example recommendations:

```
[
  {
    type: "customer_diversification",
    priority: "CRITICAL",
    text: "Reduce dependency on the largest customer by building
alternative revenue streams."
  },
  {
    type: "process_resilience",
    priority: "HIGH",
    text: "Document critical processes and assign backup
responsibility."
  },
  {
    type: "capacity_management",
    priority: "HIGH",
    text: "Increase delivery capacity or reduce backlog before scaling
demand."
  }
]
```

9. Interpretation Rules

HIGHLY_STABLE

```
The business operates with strong consistency and structural
stability.
```

STABLE_WITH_LIMITS

The business is generally stable, but some structural limits should be monitored.

FRAGILE_STRUCTURE

The business structure is fragile and exposed to disruption.

UNSTABLE

The business lacks operational stability and is exposed to disruption.

CUSTOMER_DEPENDENCY_CRITICAL

The business is critically exposed because too much revenue depends on one customer.

OPERATIONAL_SINGLE_POINT_FAILURE

The business has a single point of operational failure that could disrupt delivery.

CAPACITY_OVERLOAD

The business is operating beyond safe delivery capacity.

10. Confidence Logic

Use this order:

```
if metadata.confidenceLevel exists:
  use it
else if metadata.source === "integration":
  HIGH
```

```
else if revenueLast6Months.length >= 6:
  HIGH
else if revenueLast6Months.length >= 3:
  MEDIUM
else:
  LOW
```

Apply penalty if:

```
revenueLast6Months.length < 2
```

Confidence notes:

```
HIGH: Based on sufficient historical or integrated operational data.
MEDIUM: Based on partial revenue history and structured assumptions.
LOW: Based mainly on limited data and should be improved with more
history.
```

11. Trace Object

Include:

```
trace: {
  formulas: {
    predictabilityRatio: "standard_deviation(revenueLast6Months) /
average(revenueLast6Months)",
    stabilityScore:
      "0.30 * predictabilityScore + 0.25 * customerScore + 0.20 *
dependencyScore + 0.25 * capacityScore"
  },
  inputsUsed: input,
  scoringRules: {
    predictabilityWeight: 0.3,
    customerWeight: 0.25,
    dependencyWeight: 0.2,
    capacityWeight: 0.25
  }
}
```

12. Unit Tests

Create tests for:

Highly Stable Case

Input:

- stable revenue history
- top customer < 20%
- no key person dependency
- documented processes
- utilisation 60%

Expected:

- HIGHLY_STABLE
- high stability score

Customer Dependency Critical Case

Input:

- top customer revenue 65%

Expected:

- CUSTOMER_DEPENDENCY_CRITICAL
- SINGLE_CUSTOMER_DEPENDENCY
- CUSTOMER_CONCENTRATION_RISK

Enterprate Limited

Operational Single Point Failure Case

Input:

- keyPersonDependency true
- hasBackupForKeyPerson false

Expected:

- OPERATIONAL_SINGLE_POINT_FAILURE
- KEY_PERSON_RISK

Capacity Overload Case

Input:

- utilisation 95%
- backlog > 0

Expected:

- CAPACITY_OVERLOAD
- CAPACITY_CONSTRAINT

No Revenue History Case

Input:

- revenueLast6Months empty

Expected:

- predictabilityScore 50
- NO_REVENUE_HISTORY
- confidence penalty applied

Underutilisation Case

Input:

- utilisation 30%

Expected:

- UNDERUTILISATION_RISK

13. Acceptance Criteria

Codex should complete this when:

- Stability Engine accepts the defined input contract
- It uses computation library functions
- It returns the defined output contract
- All override classifications work
- Risk flags are generated correctly
- Recommendations are prioritised
- Interpretation is deterministic
- Confidence is included
- Trace is included
- Unit tests pass
- No database calls inside engine
- No LLM calls inside engine

14. Codex Prompt

```
Build the EnterprateAI Stability Engine in /src/lib/engines/stability.

The engine must be a pure deterministic TypeScript module. It must not
call the database, API, or LLM.
```

It should accept StabilityEngineInput and return StabilityEngineOutput.

Use the shared computation library from /src/lib/computation for:
- mean
- standard deviation
- customer concentration if needed
- weighted score

Implement:
- revenue predictability ratio
- predictability score
- customer concentration score
- operational dependency score
- capacity score
- final stability score
- override scenario classification
- risk flags
- top 3 recommendations
- interpretation
- confidence layer
- trace object

Normalize percentage inputs so both 55 and 0.55 are handled safely.

Create:
- types.ts
- rules.ts
- stability-engine.ts
- stability-engine.test.ts
- index.ts

Add unit tests for:
- highly stable case
- customer dependency critical case
- operational single point failure case
- capacity overload case
- no revenue history case
- underutilisation case

Export the engine from index.ts.